Tim Michulitz
1759024

1.

Programm

Ein Programm ist eine statische Sammlung von maschinenlesbaren Anweisungen und Daten, die als Datei auf einem Datenträger gespeichert ist.

Prozess

Ein Prozess ist die laufende Instanz eines Programms, der das Betriebssystem eigene Systemressourcen und eigenen virtuellen Adressraum im Arbeitsspeicher zuweist.

Thread

Ein Thread ist die kleinste Arbeitseinheit, die der Scheduler des Betriebssystems direkt einem Prozessorkern zur Berechnung zuweist. Ein Thread kann niemals isoliert existieren, sondern lebt immer innerhalb eines Prozesses, um die Befehle des zugrunde liegenden Programms Schritt für Schritt auszuführen.

2.

a)

Weil nicht alles von einem Programm parallel laufen kann, ist eine Beschleunigung durch mehr Kerne nicht möglich, da die Aufgabe nicht aufgeteilt werden kann (sequenziellen Anteil). Hinzu kommt der Verwaltungsaufwand (Overhead) durch Synchronisation und Kommunikation zwischen den Kernen. Bezüglich der Taktung gibt es eine Grenze bei dem Zugriff auf den Ram, da der Ram eine maximale Geschwindigkeit hat ist eine unendlich hohe Taktgeschwindigkeit bei der CPU ab einem gewissen Punkt nicht mehr sinnvoll, da die CPU sonst die meiste Zeit im Leerlauf auf Daten aus dem langsameren Arbeitsspeicher warten muss (Memory Wall).

Tim Michulitz
1759024

b)

bei einer Sekunde Zeit (für 1 Kern)

Parallelisierbarer Anteil (1-s)	Sequenzieller Anteil s	2	4	8	16
25%	s = 0,75	0,88	0,81	0,78	0,77
50%	s = 0,5	0,75	0,63	0,56	0,53
75%	s = 0,25	0,63	0,44	0,34	0,3

Rechenweg für 4 Kerne:

$$T_{MultiCore}(s,n) = T_{SingleCore}(s) + T_{SingleCore}\left(\frac{1-s}{n}\right)$$

$$T_{MultiCore}(0{,}75{,}4) = 1 \cdot (0{,}75) + 1 \cdot \left(\frac{1-0{,}75}{4}\right) = 0{,}75 + \frac{0{,}25}{4} = 0{,}75 + 0{,}0625 = 0{,}8125$$

$$T_{MultiCore}(0{,}50{,}4) = 1 \cdot (0{,}50) + 1 \cdot \left(\frac{1-0{,}50}{4}\right) = 0{,}50 + \frac{0{,}50}{4} = 0{,}50 + 0{,}125 = 0{,}625$$

$$T_{MultiCore}(0{,}25{,}4) = 1 \cdot (0{,}25) + 1 \cdot \left(\frac{1-0{,}25}{4}\right) = 0{,}25 + \frac{0{,}75}{4} = 0{,}25 + 0{,}1875 = 0{,}4375$$

3.

c)

Die absolut kürzeste Laufzeit im Testlauf wird mit drei Threads bei 472.1331 ms erreicht. Dies entspricht mehr als einer Halbierung der Dauer im Vergleich zur sequenziellen Ausführung. Ab vier Threads (594.5802 ms) steigt die benötigte Zeit wieder kontinuierlich an. Bei acht (1001.4654 ms) und sechzehn Threads (968.5373 ms) nähert sich die Laufzeit nahezu wieder dem Niveau der Single-Thread-Ausführung an.

```
$ for t in 1 2 3 4 6 8 16; do java java/Counter_beliebigviele_threads.java ./java/Tests ".*\.txt" $t; done
Anzahl Zeilen: 23291955
BenÃ¶tigte Zeit: 1103.8944 ms
Anzahl Threads: 1

Anzahl Zeilen: 23291955
BenÃ¶tigte Zeit: 652.1545 ms
Anzahl Threads: 2

Anzahl Zeilen: 23291955
BenÃ¶tigte Zeit: 472.1331 ms
Anzahl Threads: 3

Anzahl Zeilen: 23291955
BenÃ¶tigte Zeit: 594.5802 ms
Anzahl Threads: 4

Anzahl Zeilen: 23291955
BenÃ¶tigte Zeit: 716.7135 ms
Anzahl Threads: 6

Anzahl Zeilen: 23291955
BenÃ¶tigte Zeit: 1001.4654 ms
Anzahl Threads: 8

Anzahl Zeilen: 23291955
BenÃ¶tigte Zeit: 968.5373 ms
Anzahl Threads: 16
```